



Spring Framework

Training Assignment

Document Code	25e-BM/HR/HDCV/FSOFT
Version	1.1
Effective Date	20/11/2020

RECORD OF CHANGES

No	Effective Date	Change Description	Reason	Reviewer	Approver
1	26/Sep/2024	Create a new assignment	Create new		

Contents

Unit 1: Spring Core	4
Objectives:	4
Problem Descriptions:	4
Technical Requirements:	6

 	CODE	:	JSFW_M_A103
	TYPE	:	MEDIUM
	LOC	:	N/A
	DURATION	:	180 minutes

Unit 1: Spring Framework Core

Objectives:

After completing this assignment, you will:

1. Understand and practice Object-Oriented Programming (OOP) and Input/Output (IO) operations in Java.
2. Understand and apply the core concepts of the Spring Framework, such as Inversion of Control (IoC) and Dependency Injection (DI).

Problem Descriptions:

We are building the user management part of our Learning Management System (LMS). We need to model different types of users, such as Instructors (who will manage the "course-sharing" system) and Students (who will use the "assessment" system). We also need a service to save and load these user records from files.

Task 1: Employee Class

Create a base class named **LmsUser**.java in the model package that:

- Implements the Serializable interface.
- Contains the following attributes:
 1. long userId
 2. String name
 3. String email
- Includes a constructor with the above attributes as parameters.
- Implements a method void **printInfo()** that outputs the user's information in the following format: "User [ID=1, Name=John Doe, Email=john.doe@fpt.com]".
- Overrides the **toString()** method to return: "LmsUser [userId=" + userId + ", name=" + name + ", email=" + email + "]".

Task 2: Instructor Class

Create a class named **Instructor**.java in the model package that extends the **LmsUser** class:

- Contains the following additional attributes:
 1. String department
 2. String bio (A short biography)
- Includes a constructor that calls the parent LmsUser class constructor and initializes the new attributes.

- Overrides the `printlnInfo()` method to add the instructor's details. For example: "Instructor [ID=1, Name=John Doe, Email=john.doe@fpt.com, Dept=Engineering, Bio=Spring Framework Expert]".
-

Task 3: Student Class

Create a separate class named **Student.java** in the model package that:

- Implements the **Serializable** interface. (Note: This class does *not* extend `LmsUser` for this exercise).
- Contains the following attributes:
 1. long `studentId`
 2. String `name`
 3. double `gpa`
- Includes a constructor with the above attributes as parameters.
- Implements a method void **printlnInfo()** that outputs the student's information. For example: "Student [ID=101, Name=Jane Smith, GPA=3.75]".
- Overrides the **toString()** method to return: "Student [studentId=" + `studentId` + ", name=" + `name` + ", gpa=" + `gpa` + "]".

Task 4: LmsDataService Class

Create a class named **LmsDataService.java** in the service package that contains:

- A method void **saveUsers**(List<`LmsUser`> `users`) that saves a list of `LmsUser` objects (this list can contain both `LmsUser` and `Instructor` types) to a file named `users.dat`.
- A method List<`LmsUser`> **getUsers**() that reads the content of `users.dat` and returns the list of `LmsUser` objects.
- A method void **saveStudents**(List<`Student`> `students`) that saves a list of `Student` objects to a file named `students.dat`.
- A method List<`Student`> **getStudents**() that reads the content of `students.dat` and returns the list of `Student` objects.

Task 5: Test Class

Create a class named **Test.java** in the main package that uses the **LmsDataService** to perform the following:

- An attribute `LmsDataService` `dataService` for handling file operations.
- In the `main()` method:
 1. Create a list containing `LmsUser` and `Instructor` objects and save it to the file `users.dat` using the `saveUsers()` method from `dataService`.
 2. Create a list of `Student` objects and save it to the file **students.dat** using the **saveStudents()** method from `dataService`.
 3. Retrieve the saved list of `LmsUser` objects from **users.dat** and call **printlnInfo()** for each object in the list (demonstrating polymorphism).

4. Retrieve the saved list of Student objects from students.dat and call **printInfo()** for each object in the list.

Notice: You must configure Spring to create a bean for the LmsDataService class and inject that bean into the dataService attribute of the Test.jsh class (using either XML configuration or Annotation-based configuration).

Technical Requirements:

- JDK 1.8 +
- Maven 3 +
- Spring Framework Core

-- THE END --